



SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I B.Shivakalyan(192372159) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B.Shivakalyan

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and

2) Scenario:- parsing Table Conflict in LL(1).

An LL(1) parser is a down parser that reads the input from left to right and produces a leftmost derivation using one look-ahead symbol. The parsing table is constructed using the first and follow sets.

First Set:- Contains the terminals that can appear at the beginning of strings derived from a non-terminal.

Follow Set:- Contains the terminals that can appear immediately after a non-terminal.

While constructing the parsing table, each non-terminal is placed in a table entry according to the first and follow sets.

A parsing table conflict occurs when more than one production is entered into the same table for a non-terminal and an input symbol.

* left recursion

* left factoring problem

* Ambiguous grammar

Decision:-

To make the grammar suitable for LL(1) parsing Eliminate left recursion

perform Left factoring.

Ensure first and follow sets do not overlap.

After these modifications each parsing table entry containing any one production resulting deterministic and conflict free parsing

2) Scenario:- Incorrect parse tree generation

A Syntax analyzer constructs a parse tree by applying grammar production according to the input tokens. The parse tree represents the syntactic structure of the expression

if the grammar does not define associating the parser may generate the parse tree

$id + (id - id)$

instead of $(id + id) - id$

Since addition and subtraction are left-associative the second interpretation is correct

Decision:-

Modify the grammar to enforce left associating

grammar:-

$E \rightarrow E + T / E - T / T$

$T \rightarrow id$

This grammar ensure expression one evaluated from left to right producing the correct parse tree & correct evaluation

3) Scenario:- invalid expression handling

The lexical analyzer converts the input:

$a * + b$

into valid tokens

identifier (a)

operator (+)

operator (*)

identifier (b)

After the '+' operator the grammar expects an operand (identifier or expression) instead another operator '*' appears

Therefore the parser cannot construct and reports a syntax error

in predictive parsing (LL parser):

- The parser checker & the parsing table.

- Since no valid production exists it reports an error

in shift-reduce parsing (LR parser)

- The parser checks the Action Table, without recovery parsing stops immediately

Decision:-

To improve parsing:-

- Use panic mode Recovery
- Use phrase-level Recovery
- Use Synchronizing Tokens based on Follow sets
- Display meaningful error message.

These techniques help the parser continue parsing the remaining and improve compiler error reporting

4) Scenario:- Ambiguity in Conditional Statement

Given grammar

$E \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$

This grammar is ambiguous because the parser cannot determine to which of the statements the else belongs.

This ambiguity is to which statement the else belongs.

Productions begin with the same prefix $\text{if } E \text{ then}$.

Scenario :- operator precedence Conflict

Given grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow id$$

This grammar is ambiguous it does not define operator Precedence or associativity for the expression

$$id + id * id$$

Two parse trees are possible

1. $(id + id) * id$

2. $id + (id * id)$

The Correct evaluation is

$$id + (id * id)$$

because multiplication has higher precedence than addition without precedence.

During parsing the parser cannot decide whether to

Reduce using $E \rightarrow id$ if E then 1

Continue using $E \rightarrow E + E$ if E then 2 else 3

This evaluates parsing condition and multiple possible parse trees